

Multimedia Application

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University Tashkent.

Email: minhaz.ahmed@gmail.com

Content

- ▶ Naive Bayes Classifiers
- ▶ Training the Naive Bayes Classifier
- ▶ Worked example
- ▶ Optimizing for Sentiment Analysis
- ▶ Naive Bayes for other text classification tasks
- ▶ Naive Bayes as a Language Model

Bayes theorem

- ▶ Bayes' theorem is also known as **Bayes' Rule** or **Bayes' law**, which is used to determine the probability of a hypothesis with prior knowledge. It depends on the conditional probability

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

P(A|B) is Posterior probability: Probability of hypothesis A on the observed event B.

P(B|A) is Likelihood probability: Probability of the evidence given that the probability of a hypothesis is true.

P(A) is Prior Probability: Probability of hypothesis before observing the evidence.

P(B) is Marginal Probability: Probability of Evidence.

Types Of Naive Bayes:

- ▶ There are three types of Naive Bayes model under the scikit-learn library:
 - Gaussian: It is used in classification and it assumes that features follow a normal distribution.
 - Multinomial: It is used for discrete counts. For example, let's say, we have a text classification problem. Here we can consider Bernoulli trials which is one step further and instead of “word occurring in the document”, we have “count how often word occurs in the document”, you can think of it as “number of times outcome number x_i is observed over the n trials”.
 - Bernoulli: The binomial model is useful if your feature vectors are binary (i.e. zeros and ones). One application would be text classification with ‘bag of words’ model where the 1s & 0s are “word occurs in the document” and “word does not occur in the document” respectively.

Example

Consider the given Dataset ,Apply Naive Baye's Algorithm and Predict that if a fruit has the following properties then which type of the fruit it is
Fruit = {Yellow , Sweet ,long}

Frequency Table:

Fruit	Yellow	Sweet	Long	Total
Mango	350	450	0	650
Banana	400	300	350	400
Others	50	100	50	150
Total	800	850	400	1200

Example solution

▶ Solution:

▶ $P(A|B) = (P(B|A) * P(A)) / P(B)$

Mango:

▶ $P(X | \text{Mango}) = P(\text{Ye} | \text{Yellow}) * P(\text{Sweet} | \text{Mango}) * P(\text{Long} | \text{Mango})$

a) $P(\text{Yellow} | \text{Mango}) = (P(\text{Mango} | \text{Yellow}) * P(\text{Yellow})) / P(\text{Mango})$

▶ $= ((350/800) * (800/1200)) / (650/1200)$

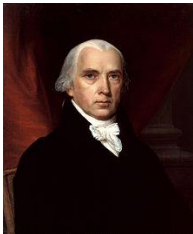
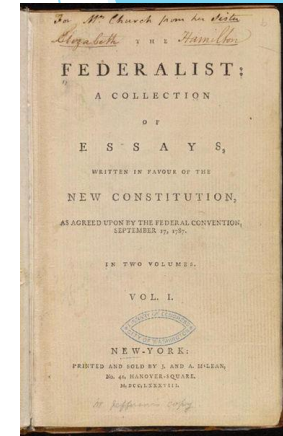
▶ $P(\text{Yellow} | \text{Mango}) = 0.53 \rightarrow 1$

Text Classification

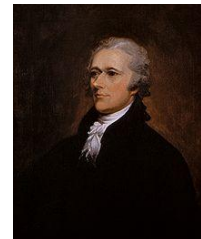
- ▶ Assigning subject categories, topics, or genres
- ▶ Spam detection
- ▶ Authorship identification
- ▶ Age/gender identification
- ▶ Language Identification
- ▶ Sentiment analysis

Who wrote which Federalist papers?

- ▶ 1787-8: anonymous essays try to convince New York to ratify U.S Constitution: Jay, Madison, Hamilton.
- ▶ Authorship of 12 of the letters in dispute
- ▶ 1963: solved by Mosteller and Wallace using Bayesian methods



James Madison



Alexander Hamilton

Male or female author from a given text

By 1925 present-day Vietnam was divided into three parts under French colonial rule. The southern region embracing Saigon and the Mekong delta was the colony of Cochinchina; the central area with its imperial capital at Hue was the protectorate of Annam ...

Clara never failed to be astonished by the extraordinary felicity of her own name. She found it hard to trust herself to the mercy of fate, which had managed over the years to convert her greatest shame into one of her greatest assets...

Text Classification: definition

- ▶ *Input:*

- ▶ a document d
- ▶ a fixed set of classes $C = \{c_1, c_2, \dots, c_J\}$

- ▶ *Output:* a predicted class $c \in C$

Classification Methods:

Hand-coded rules

- ▶ Rules based on combinations of words or other features
 - ▶ spam: black-list-address OR (“dollars” AND “have been selected”)
- ▶ Accuracy can be high
 - ▶ If rules carefully refined by expert
- ▶ But building and maintaining these rules is expensive

Classification Methods: Supervised Machine Learning

► *Input:*

- a document d
- a fixed set of classes $C = \{c_1, c_2, \dots, c_J\}$
- A training set of m hand-labeled documents $(d_1, c_1), \dots, (d_m, c_m)$

► *Output:*

- a learned classifier $\gamma: d \rightarrow c$

Classification Methods: Supervised Machine Learning

- ▶ Any kind of classifier
 - ▶ Naïve Bayes
 - ▶ Logistic regression
 - ▶ Support-vector machines
 - ▶ k-Nearest Neighbors

Naive Bayes Intuition

- ▶ Simple ("naive") classification method based on Bayes rule
- ▶ Relies on very simple representation of document
 - ▶ **Bag of words**

The Bag of Words Representation

- ▶ We preprocess the dataset by converting each email into a bag-of-words representation, where each word is a feature and its frequency in the email is its value. We also assign a label (spam or not spam) to each email

The Bag of Words Representation

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!



it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
...	...

The bag of words representation

$$Y(\text{table}) = C$$

seen	2
sweet	1
whimsical	1
recommend	1
happy	1
...	...

Training

- ▶ We train the Naïve Bayes classifier on the labeled dataset. During training, the classifier calculates the probabilities of each word occurring in spam and not spam emails, as well as the prior probabilities of spam and not spam emails in the dataset.

Prediction

Step 1: Given a new email, we convert it into a bag-of-words representation.

Step 2: For each word in the email, we calculate its conditional probability of occurring in spam and not spam emails based on the probabilities learned during training.

Step 3: We multiply the conditional probabilities of all words in the email and multiply them by the prior probabilities of spam and not spam emails.

Step 4: We compare the calculated probabilities for spam and not spam, and classify the email as spam or not spam based on the higher probability.

Bayes' Rule Applied to Documents and Classes

- ▶ For a document d and a class c

$$P(c | d) = \frac{P(d | c)P(c)}{P(d)}$$

Naive Bayes Classifier (I)

$$c_{MAP} = \operatorname{argmax}_{c \in C} P(c | d)$$

MAP is “maximum a posteriori” = most likely class

$$= \operatorname{argmax}_{c \in C} \frac{P(d | c)P(c)}{P(d)}$$

Bayes Rule

$$= \operatorname{argmax}_{c \in C} P(d | c)P(c)$$

Dropping the denominator

Text Classification and Naïve Bayes

Naive Bayes: Learning

Learning the Multinomial Naive Bayes Model

- ▶ First attempt: maximum likelihood estimates
 - ▶ simply use the frequencies in the data

$$\hat{P}(c_j) = \frac{N_{c_j}}{N_{total}}$$

$$\hat{P}(w_i | c_j) = \frac{\text{count}(w_i, c_j)}{\sum_{w \in V} \text{count}(w, c_j)}$$

Parameter estimation

$$\hat{P}(w_i | c_j) = \frac{\text{count}(w_i, c_j)}{\sum_{w \in V} \text{count}(w, c_j)}$$

fraction of times word w_i appears
among all words in documents of topic c_j

- ▶ Create mega-document for topic j by concatenating all docs in this topic
 - ▶ Use frequency of w in mega-document

Problem with Maximum Likelihood

- ▶ What if we have seen no training documents with the word ***fantastic*** and classified in the topic **positive** (*thumbs-up*)?

$$\hat{P}(\text{"fantastic"} \mid \text{positive}) = \frac{\text{count}(\text{"fantastic"}, \text{positive})}{\sum_{w \in V} \text{count}(w, \text{positive})} = 0$$

- ▶ Zero probabilities cannot be conditioned away, no matter the other evidence!

$$c_{MAP} = \operatorname{argmax}_c \hat{P}(c) \prod_i \hat{P}(x_i \mid c)$$

Laplace (add-1) smoothing for Naïve Bayes

$$\hat{P}(w_i | c) = \frac{\text{count}(w_i, c)}{\sum_{w \in V} \text{count}(w, c)}$$

$$= \frac{\text{count}(w_i, c) + 1}{\sum_{w \in V} \text{count}(w, c) + |V|}$$

Multinomial Naïve Bayes: Learning

- ▶ From training corpus, extract *Vocabulary*

- ▶ Calculate $P(c_j)$ terms

- ▶ For each c_j in C do

$docs_j \leftarrow$ all docs with class $= c_j$

$$P(c_j) \propto \frac{|docs_j|}{|\text{total \# documents}|}$$

- Calculate $P(w_k | c_j)$ terms

- $Text_j \leftarrow$ single doc containing all $docs_j$

- For each word w_k in *Vocabulary*

$n_k \leftarrow$ # of occurrences of w_k in $Text_j$

$$P(w_k | c_j) \propto \frac{n_k + \alpha}{n + \alpha |Vocabulary|}$$

Unknown words

- ▶ What about unknown words
 - ▶ that appear in our test data
 - ▶ but not in our training data or vocabulary?
- ▶ We **ignore** them
 - ▶ Remove them from the test document!
 - ▶ Pretend they weren't there!
 - ▶ Don't include any probability for them at all!
- ▶ Why don't we build an unknown word model?
 - ▶ It doesn't help: knowing which class has more unknown words is not generally helpful!

Stop words

- ▶ Some systems ignore stop words
 - ▶ **Stop words:** very frequent words like *the* and *a*.
 - ▶ Sort the vocabulary by word frequency in training set
 - ▶ Call the top 10 or 50 words the **stopword list**.
 - ▶ Remove all stop words from both training and test sets
 - ▶ As if they were never there!
- ▶ But removing stop words doesn't usually help
 - So in practice most NB algorithms use **all** words and **don't** use stopwords lists

Naive Bayes: Learning

Sentiment Example:

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

A worked sentiment example with add-1 smoothing

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

1. Prior from training:

$$\hat{P}(c_j) = \frac{N_{c_j}}{N_{total}} \quad \begin{array}{l} P(-) = 3/5 \\ P(+) = 2/5 \end{array}$$

2. Drop "with"

3. Likelihoods from training:

$$p(w_i|c) = \frac{\text{count}(w_i, c) + 1}{(\sum_{w \in V} \text{count}(w, c)) + |V|}$$

$$P(\text{"predictable"}|-) = \frac{1+1}{14+20} \quad P(\text{"predictable"}|+) = \frac{0+1}{9+20}$$

$$P(\text{"no"}|-) = \frac{1+1}{14+20} \quad P(\text{"no"}|+) = \frac{0+1}{9+20}$$

$$P(\text{"fun"}|-) = \frac{0+1}{14+20} \quad P(\text{"fun"}|+) = \frac{1+1}{9+20}$$

4. Scoring the test set:

$$P(-)P(S|-) = \frac{3}{5} \times \frac{2 \times 2 \times 1}{34^3} = 6.1 \times 10^{-5}$$

$$P(+)P(S|+) = \frac{2}{5} \times \frac{1 \times 1 \times 2}{29^3} = 3.2 \times 10^{-5}$$

Optimizing for sentiment analysis

For tasks like sentiment, word **occurrence** seems to be more important than word **frequency**.

- ▶ The occurrence of the word *fantastic* tells us a lot
- ▶ The fact that it occurs 5 times may not tell us much more.

Binary multinominal naive bayes, or binary NB

- ▶ Clip our word counts at 1

Binary Multinomial Naïve Bayes: Learning

- From training corpus, extract *Vocabulary*
- ▶ Calculate $P(c_j)$ terms
 - ▶ For each c_j in C do
 - $docs_j \leftarrow$ all docs with class $= c_j$
$$P(c_j) \propto \frac{|docs_j|}{|\text{total \# documents}|}$$
- Calculate $P(w_k | c_j)$ terms
 - $Text_j \leftarrow$ single doc containing all $docs_j$
 - For each word w_k in *Vocabulary*
 - $n_k \leftarrow$ # of occurrences of w_k in $Text_j$
 - Remove duplicates in each doc:
 - For each word type w in doc_j
 - Retain only a single instance of w
$$P(w_k | c_j) \propto \frac{n_k + \alpha}{n + \alpha |Vocabulary|}$$

Binary Multinomial Naive Bayes on a test document d

First remove all duplicate words from d

Then compute NB using the same equation

$$c_{NB} = \operatorname{argmax}_{c_j \in C} P(c_j) \prod_{i \in \text{positions}} P(w_i | c_j)$$

Binary multinominal naive Bayes

Four original documents:

- it was pathetic the worst part was the boxing scenes
- no plot twists or great scenes
- + and satire and great plot twists
- + great scenes great film

After per-document binarization:

- it was pathetic the worst part boxing scenes
- no plot twists or great scenes
- + and satire great plot twists
- + great scenes film

	NB Counts		Binary Counts	
	+	–	+	–
and	2	0	1	0
boxing	0	1	0	1
film	1	0	1	0
great	3	1	2	1
it	0	1	0	1
no	0	1	0	1
or	0	1	0	1
part	0	1	0	1
pathetic	0	1	0	1
plot	1	1	1	1
satire	1	0	1	0
scenes	1	2	1	2
the	0	2	0	1
twists	1	1	1	1
was	0	2	0	1
worst	0	1	0	1

Binary multinomial naive Bayes

Four original documents:

- it was pathetic the worst part was the boxing scenes
- no plot twists or great scenes
- + and satire and great plot twists
- + great scenes great film

After per-document binarization:

- it was pathetic the worst part boxing scenes
- no plot twists or great scenes
- + and satire great plot twists
- + great scenes film

	NB Counts		Binary Counts	
	+	–	+	–
and	2	0	1	0
boxing	0	1	0	1
film	1	0	1	0
great	3	1	2	1
it	0	1	0	1
no	0	1	0	1
or	0	1	0	1
part	0	1	0	1
pathetic	0	1	0	1
plot	1	1	1	1
satire	1	0	1	0
scenes	1	2	1	2
the	0	2	0	1
twists	1	1	1	1
was	0	2	0	1
worst	0	1	0	1

Counts can still be 2! Binarization is within-doc!

More on Sentiment Classification

- ▶ I really like this movie
I really **don't** like this movie

Negation changes the meaning of "like" to negative.

Negation can also change negative to positive-ish

- **Don't** dismiss this film
- **Doesn't** let us get bored

Sentiment Classification: Lexicons

Sometimes we don't have enough labeled training data

In that case, we can make use of pre-built word lists

Called **lexicons**

There are various publicly available lexicons

MPQA Subjectivity Cues Lexicon

Theresa Wilson, Janyce Wiebe, and Paul Hoffmann (2005). Recognizing Contextual Polarity in Phrase-Level Sentiment Analysis. Proc. of HLT-EMNLP-2005.

Riloff and Wiebe (2003). Learning extraction patterns for subjective expressions. EMNLP-2003.

- ▶ Home page: https://mpqa.cs.pitt.edu/lexicons/subj_lexicon/
- ▶ 6885 words from 8221 lemmas, annotated for intensity (strong/weak)
 - ▶ 2718 positive
 - ▶ 4912 negative
- ▶ + : *admirable, beautiful, confident, dazzling, ecstatic, favor, glee, great*
- ▶ – : *awful, bad, bias, catastrophe, cheat, deny, envious, foul, harsh, hate*

Using Lexicons in Sentiment Classification

Add a feature that gets a count whenever a word from the lexicon occurs

- ▶ E.g., a feature called "this word occurs in the positive lexicon" or "this word occurs in the negative lexicon"

Now all positive words (*good, great, beautiful, wonderful*) or negative words count for that feature.

Using 1-2 features isn't as good as using all the words.

- But when training data is sparse or not representative of the test set, dense lexicon features can help

Naive Bayes in Other tasks: Spam Filtering

- ▶ Spam Assassin Features:
 - ▶ Mentions millions of (dollar) ((dollar) NN,NNN,NNN.NN)
 - ▶ From: starts with many numbers
 - ▶ Subject is all capitals
 - ▶ HTML has a low ratio of text to image area
 - ▶ "One hundred percent guaranteed"
 - ▶ Claims you can be removed from the list

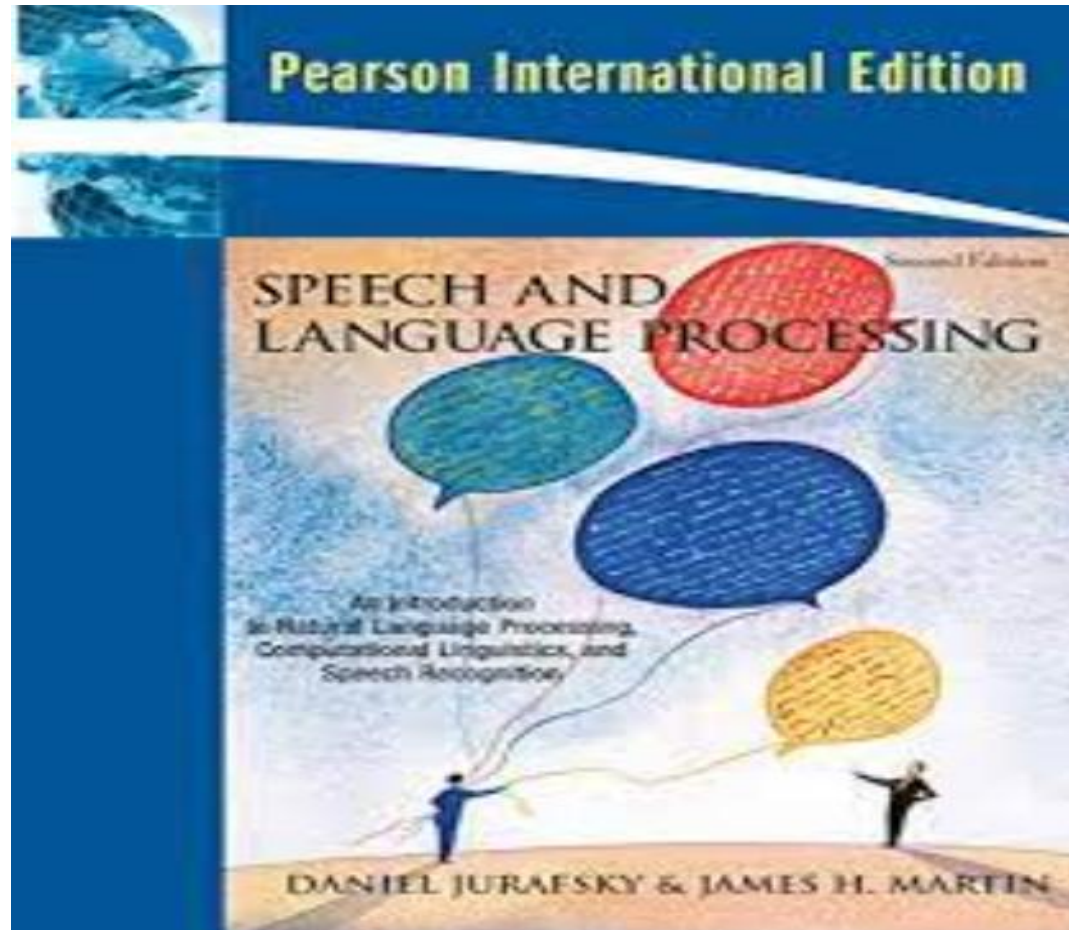
Naive Bayes in Language ID

- ▶ Determining what language a piece of text is written in.
Features based on character n-grams do very well
- ▶ Important to train on lots of varieties of each language
(e.g., American English varieties like African-American English, or English varieties around the world like Indian English)

Summary: Naive Bayes is Not So Naive

- ❑ Very Fast, low storage requirements
- ❑ Work well with very small amounts of training data
- ❑ Robust to Irrelevant Features
 - ❑ Irrelevant Features cancel each other without affecting results
- ❑ Very good in domains with many equally important features
 - ❑ Decision Trees suffer from *fragmentation* in such cases – especially if little data
- ❑ Optimal if the independence assumptions hold: If assumed independence is correct, then it is the Bayes Optimal Classifier for problem
- ❑ A good dependable baseline for text classification

Reference



Chapter 4

Question



Thank you

